OFFICIAL PROJECT BRIEFING · OCTOBER 2026

FarmRescue

Emergency Agricultural Early-Warning & Response Platform

"What should I do when the farm suddenly goes wrong?"

Deterministic agronomic risk intelligence that turns raw field evidence into validated, expert-authored action plans in seconds — where data and rules decide, and AI only explains.

Next.js 14 (App Router) · TypeScript · Tailwind CSS · MongoDB · Gemini explanation layer

Hackathon Product Demo · Prototype Grade: Demo-Ready

Executive Summary

The problem

Crop-loss emergencies unfold in hours, but trusted, location-specific agronomic guidance reaches farmers in days.

The solution

A field-evidence early-warning platform that assesses a reported problem in seconds and returns a specific, safety-checked response plan.

The difference

Data and validated rules decide risk and allowed actions. AI only explains validated results — it never overrides a classification or invents a plan.

Implementation at a glance

31
automated tests passing (zero-dependency Node runner)

12
expert-authored action plans (ICAR / IRRI-aligned)

19
documented REST API routes

0
decisions authored by the AI layer

The Problem

A crop emergency does not wait for the extension officer.

Waterlogging, heat stress and outbreaks escalate within a single day — the window to act is measured in hours.

Invisible onset

Waterlogging, submergence and heat stress cross critical thresholds silently overnight. By the time damage is visible, the highest-value intervention window has already closed.

Generic, sometimes dangerous advice

Blanket advisory (apply urea, spray on schedule) is unsafe under active flooding. Advice that ignores field state can make the loss worse — e.g. granular urea in standing water.

No traceability

Advice arrives by word of mouth with no record of the evidence used, who verified it, or whether the crop recovered. Nothing is auditable, so nothing improves.

The agronomic knowledge exists — ICAR / IRRI protocols are well documented. The failure is delivery: it does not reach the farmer at the moment of need, in a form that is actionable and safe.

DESIGN PRINCIPLE

The Non-Negotiable Invariant

> “Data and validated agricultural rules determine risk and allowed actions. AI explains validated results; AI does not independently decide what a farmer should do.”

✅ The deterministic core

- Versioned rule set evaluates evidence → severity.
- Pre-stored action library keyed by ruleFamily::riskLevel.
- Safety gate validates every plan before display.
- Same inputs → identical plan, every single time.

🤖 The AI layer (subordinate)

- Gemini runs strictly after risk + safety decisions.
- Strict prompt boundary: explain only, never classify.
- Cannot alter severity, actions or safety interceptions.
- Deterministic fallback — the app works with Gemini offline.

Consequence: the platform stays useful, auditable and defensible even when the model is unavailable, wrong or over-eager.

Architecture & Decision Pipeline

1. **Inputs**
   Live weather (Open-Meteo + calibrated agro-climatic fallback), farmer observations, ground measurements.

2. **Evidence Engine**
   Normalizes every signal into typed evidence carrying source, strength, reliability and freshness.

3. **Deterministic Risk Engine**
   Versioned rule set maps evidence → severity (NORMAL → CRITICAL) with no randomness.

4. **Safety Gate**
   Validates the planned actions and injects AVOID interceptions for hazardous interventions.

5. **Action Plan Library**
   Returns a pre-stored expert plan: DO NOW, DO NEXT, AVOID, MONITOR, ESCALATE.

6. **Explain & Verify**
   AI explains the validated result; field workers verify ground truth; outcome is logged.

Persistence layer · MongoDB hydrate-on-boot, write-behind (debounced ≈800 ms) · transparent in-memory fallback if the database is unreachable · live health at GET /api/system/persistence

Roles & Personas

FARMER

Report incidents; view farm status, live weather and active incidents; follow the validated plan; add ground measurements; run the scenario simulator; submit outcome feedback.

Screens: / /incidents/new /incidents/:id /history

FIELD_WORKER

See a queue of incidents flagged for verification, with distance indicators; submit ground-truth verification and optionally override severity.

Screens: /field-worker

FPO_ADMIN

Planned role for FPO-level coordination — community drainage canals, pump support, seed and nursery logistics. Today it appears only in escalation text.

Screens: — (planned)

SYSTEM_ADMIN

Inspect persistence and platform health via the system API.

Screens: GET /api/system/persistence

Demo personas: Krishna Reddy — farmer, 2.8-acre paddy (“Vemuru Riverbed Field”, BPT 5204 / Samba Mahsuri, tillering, poor drainage). Ramesh Kumar — agricultural field officer.

Core Capabilities

1

Farmer Dashboard

Live 24h rainfall, temperature, humidity and wind with active incidents and severity badges.

2

Emergency Reporting Wizard

Non-academic 3-step flow: condition, symptom chips, evidence confirmation.

3

Hero Action Screen

DO NOW / AVOID / MONITOR, why-this-recommendation, evidence traceability, live reassessment.

4

Field Worker Queue

Human-in-the-loop verification: confirm, partially confirm, reject or request more information.

5

Auditable History

Non-destructive timeline preserving every assessment version, action plan and outcome.

6

Scenario Simulator

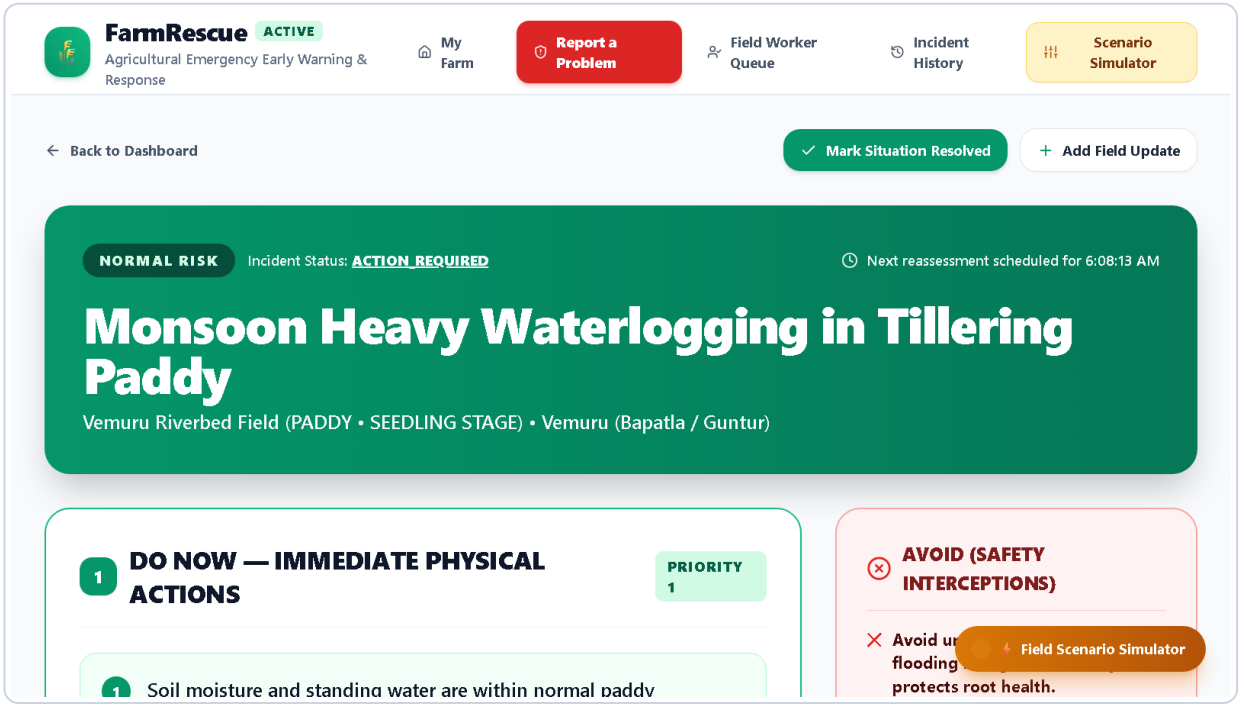
Manipulate rainfall, duration, depth, drainage and crop stage → instant re-assessment everywhere.

Plus, on every screen:

- Pre-stored recommendation library — 12 expert plans, delivered instantly (no generation step, identical on every recurrence).
- Optional durable persistence — set MONGODB_URI and restarts survive; otherwise the app runs transparently in memory.

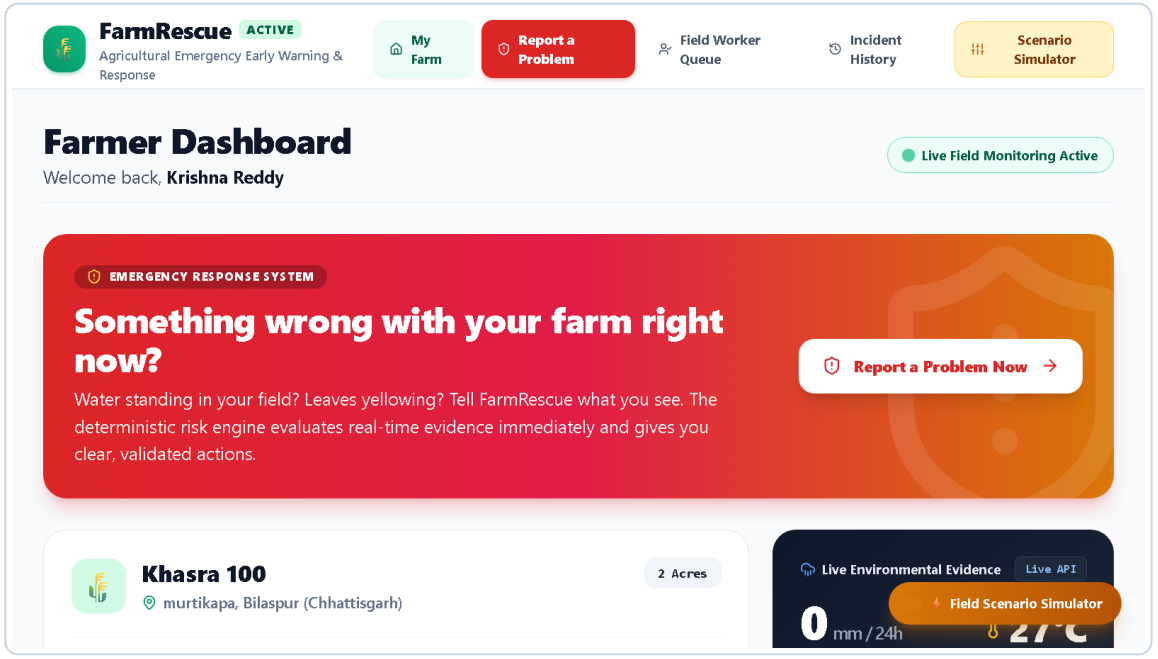
The Emergency Action Screen

- **DO NOW**
- Step-by-step immediate physical interventions (e.g. cut the drainage breach at the lowest bund corner).
- **AVOID (safety interceptions)**
- Harmful actions the Safety Gate blocks — e.g. no granular urea into standing flood water, no contact pesticide sprays.
- **MONITOR**
- Symptoms and submergence thresholds to watch, with a review deadline.
- **WHY THIS RECOMMENDATION?**
- Plain-language explanation generated strictly from the validated result.
- **Evidence view**
- Traceability cards showing source, reliability and freshness (🟢 Fresh 🟡 Aging 🔴 Stale).
- **Reassess or resolve**
- Add new ground measurements for live recalculation, then submit the outcome.



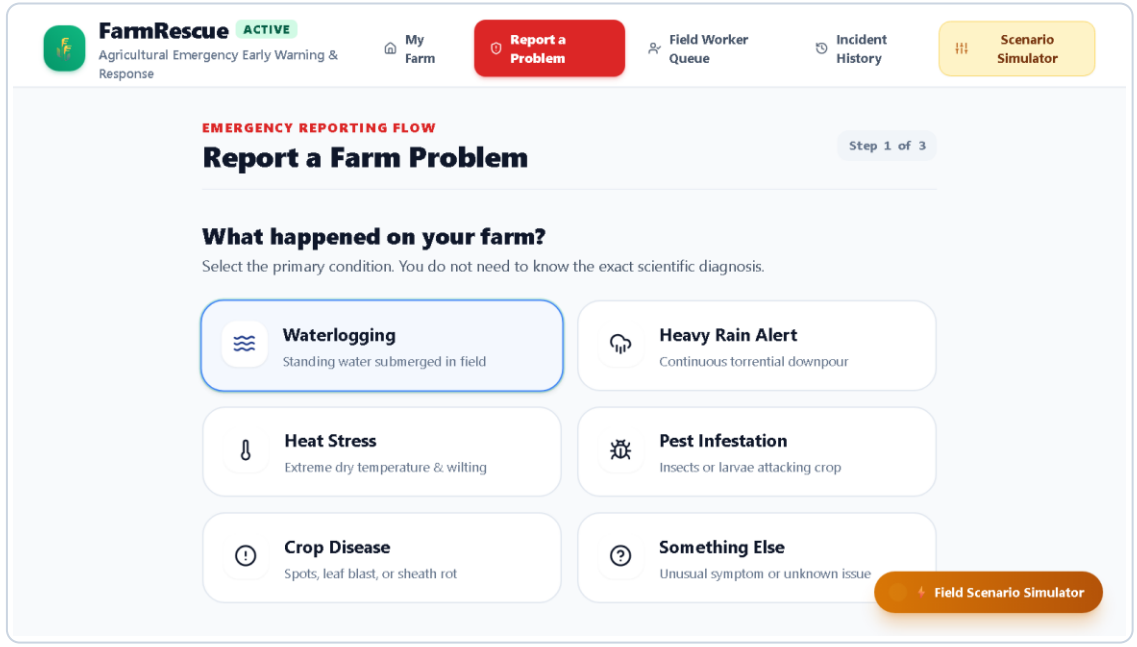
Incident response screen — /incidents/:id (live build)

Dashboard & Emergency Reporting



Farmer Dashboard — /

Live environmental conditions, active incidents with severity badges, and a prominent emergency CTA: “Something wrong with your farm right now?”



Reporting Wizard — /incidents/new

Three steps: what happened, quick symptom chips (e.g. “water standing > 15 cm”), then “Let’s check what we know” before the risk engine runs.

Deterministic Risk Engine

Paddy waterlogging thresholds (as implemented)

CRITICAL	rainfall ≥ 100 mm OR (rainfall ≥ 75 mm AND standing water ≥ 36 h) — both require poor drainage	**Escalates**
HIGH	rainfall ≥ 60 mm OR standing water ≥ 24 h OR depth ≥ 15 cm OR rainfall ≥ 50 mm in a sensitive stage	**Escalates**
MODERATE	rainfall ≥ 30 mm OR standing water ≥ 8 h	No escalation
LOW	rainfall ≥ 15 mm OR any standing water > 0 h	No escalation
NORMAL	otherwise	No escalation

Rule-priority policy: an explicitly reported incident type always routes to its dedicated rule. Ambient signals (heavy rain, standing water, extreme heat) only auto-route 'Something else' reports — so a heat-stress report can never be hijacked into a waterlogging plan.

Review cadence: reviewAt = assessedAt + { CRITICAL: 3 h, HIGH: 6 h, otherwise: 12 h }.

Recommendation Library & Safety Gate

12 pre-stored expert action plans

- Waterlogging — 5 severities (NORMAL → CRITICAL).
- Heat stress and pest / disease cover every severity the engine emits.
- Keyed by ruleFamily::riskLevel — plans appear instantly, with no generation step.
- Identical plan every time the same conditions recur (deterministic).

Safety Gate

- Validates every plan before it is displayed.
- Blocks hazardous interventions outright (AVOID).
- Escalates to a human when evidence is thin or conflicting.
- The engine never recommends a pesticide — verified diagnosis first.

Representative interceptions

DO NOT apply granular urea into standing flood water — it is lost and worsens the anaerobic condition.

DO NOT spray contact pesticides on a water-stressed or submerged canopy — phytotoxicity risk.

DO NOT re-transplant or top-dress until standing water has receded and the review deadline has passed.

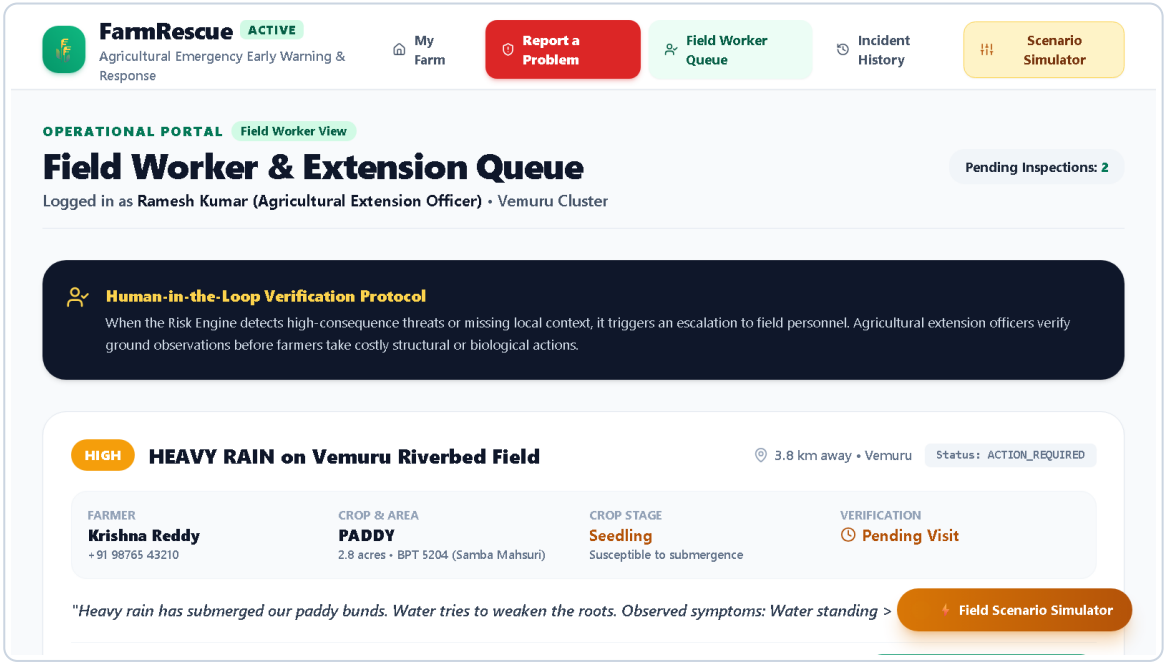
Evidence Quality & Human Verification

Every assessment is traceable

- Each observation carries source, reliability, strength and timestamps (observedAt / retrievedAt / freshnessMinutes).
- Freshness is surfaced, never hidden: Fresh Aging Stale.
- Observed vs forecast values are labelled distinctly — no silent blending.
- Unified evidence view returns verdicts plus what is still missing.

Human-in-the-loop escalation

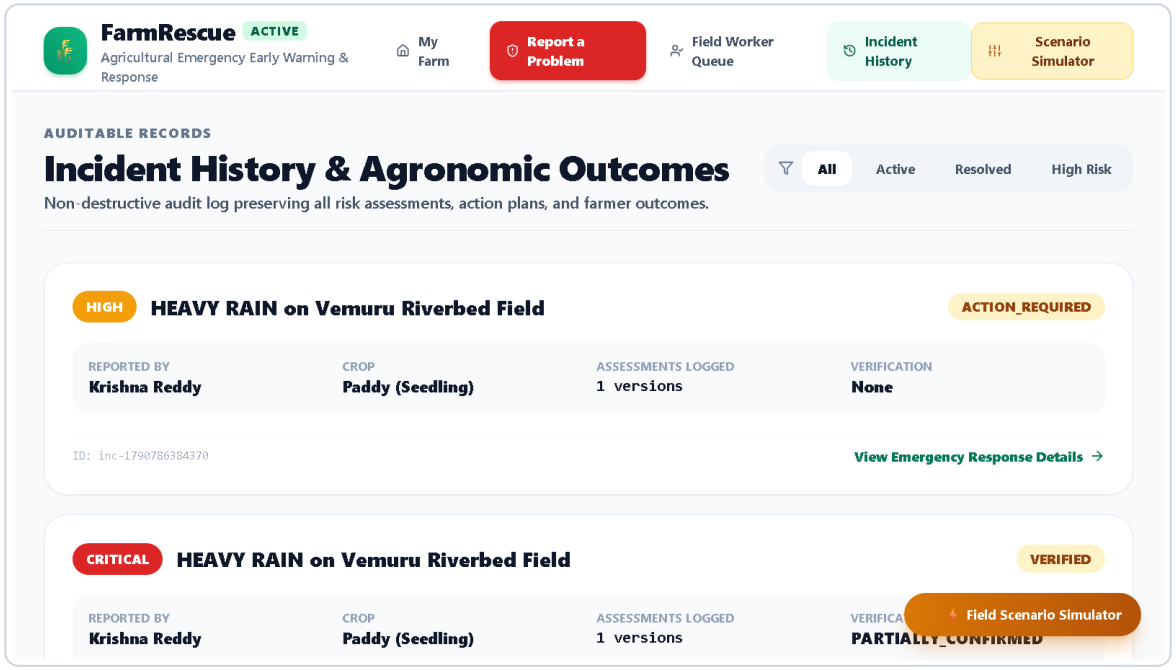
- HIGH / CRITICAL incidents auto-escalate to the field-worker queue.
- Officer confirms, partially confirms, rejects or requests more information, and may override severity.
- Verified ground truth is appended to the audit trail, not overwritten.



Field Worker & Extension Queue — /field-worker

Pending inspections with distance indicators and a structured verification form.

Persistence, Degradation & Audit



Auditable History — /history

Non-destructive audit log preserving every assessment version, action plan, verification and farmer outcome.

Durable by default, graceful by design

- Set MONGODB_URI and every incident / farm mutation is persisted.
- Write-behind, debounced ≈800 ms after each mutation.
- Boot hydrates from the database — restarts survive.
- If the database is unreachable the app transparently runs in memory and retries in the background.
- Live health: GET /api/system/persistence.
- Reassessments append; nothing is destroyed or silently rewritten.

Technology Stack

Application

- Next.js 14 — App Router
- TypeScript (strict)
- Tailwind CSS
- Lucide React icons

Data & Decisions

- Versioned deterministic rule engine
- 12-plan expert action library
- Evidence freshness & verdict pipeline
- Safety gate interceptions

External Services

- Open-Meteo live weather API
- Calibrated agro-climatic fallback model
- Google Gemini — explanation layer only
- MongoDB — optional durable store

Quality

- Zero-dependency Node test runner
- 31 tests passing
- Strict prompt boundary on the AI layer
- ICAR / IRRI-aligned agronomic content

No exotic dependencies, no paid services required to run the demo — GEMINI_API_KEY and MONGODB_URI are both optional.

Primary Demonstration Script

1. **Baseline**
   Krishna Reddy's paddy field in tillering faces 82 mm of rain and 24 h of standing water → HIGH RISK.

2. **Review the plan**
   Inspect DO NOW (breach bunds, clear drainage), AVOID (urea application blocked) and Why This Recommendation.

3. **Field verification**
   Open the Field Worker Queue, inspect the incident and submit official verification.

4. **Simulate escalation**
   In the Scenario Simulator push rainfall to 115 mm and duration to 48 h → instant CRITICAL RISK.

5. **Simulate de-escalation**
   Drop rainfall to 5 mm, standing water to 0 h and set drainage to good → LOW / NORMAL.

6. **Report and persist**
   Report a new incident through the wizard and watch the deterministic engine assess it live; restart the server to prove persistence.

Run the demo from a single dev server. Only one process should own the shared Atlas data at a time.

Parked for the Demo Build

⚠ **Field Map (/field) and Plot Registry (/plots) are disabled in the demo build — they return 404 and are not linked in the navigation.**

Why they were parked

- The registry could only find plots that had been registered inside FarmRescue — there is no government land-records verification, so it made a promise the data could not keep.
- Plot coordinates came from user-placed map taps or GPS, not cadastral survey data.
- Two concurrent dev servers overwrote each other in the shared database, making the feature look broken in front of evaluators.

Status and path forward

- The full implementation is preserved, not deleted — cascading State → Village → Khasra lookup, map chips, register-on-the-spot, Bhuiyan-style khasra report entry.
- Restore is a one-step move from the parked folder.
- Revisit requires a real land-records API — a Bhulekh / ULPIN-style integration, plus a single-writer data policy.

Decision: show a smaller honest product rather than a larger one that cannot be defended in a live demo.

Known Gaps & Roadmap

Known gaps (honest list)

- No authentication — roles exist in the type system and UI, but API routes accept no credentials.
- Recommendation library is paddy-centric for waterlogging; other crops fall through to the generic plan.
- Incident creation seeds a fixed 65 mm rainfall reading; standing water defaults to 12 h / 10 cm.
- “Next reassessment scheduled” is display-only — no background scheduler exists.
- Test coverage is partial: risk boundaries, safety-gate rules and library coverage are still untested.
- No real SMS / WhatsApp / IVR delivery — notification exists only as escalation text.

Roadmap

P1 · Hardening

Boundary tests for risk + safety rules; a real scheduled reassessment loop; live weather evidence on incident creation; basic role authentication.

P2 · Breadth

Multi-crop action plans; photo evidence capture → storage → verification; more agro-climatic regions.

P3 · Depth

Offline-first PWA; sensor / IoT intake; outcome learning loop; multi-tenancy for FPOs; land-records integration.

The P0 MVP is complete and demo-ready. Everything above is scheduled work, not a hidden failure.

CLOSING

When the farm goes wrong at 3 a.m., FarmRescue already knows what to do.

A deterministic agronomic decision engine, an expert-authored response library, a safety gate that blocks harmful advice, and a human verification loop — with AI strictly in the role of explainer.

Thank you · Questions welcome

FarmRescue — Emergency Agricultural Early-Warning & Response Platform · October 2026